ALLIANCE UNIVERSITY · BCA (AI/ML) · DBMS ASSIGNMENT SERIES

06

ASSIGNMENT 6

Data Integrity Constraints

Online Food Delivery System

SUBMITTED BY
Vimal Kumar Rao

PROGRAMME
BCA (AI/ML)

INSTITUTION
Alliance University

SUBMISSION DATE
24 October 2026

IN THIS ASSIGNMENT

01 Define rules

02 Apply constraints

03 Verify data

04 Test failures

A consistent relational design carried forward from Assignments 1 and 2.

1 Objective and Brief Coverage

Constraints are rules that the **database itself** enforces, so that invalid data is rejected no matter which application, script or person tries to insert it. Primary and foreign keys were created in Assignment 5; this stage adds **UNIQUE**, **CHECK** and **DEFAULT** rules, explains each one, and proves with queries that the sample data respects them. The script targets **MySQL 8.0.16 or later**; earlier versions parse CHECK but ignore it.

8	9	6	10	7
primary keys	foreign keys	UNIQUE rules	CHECK rules	DEFAULT values

	Brief requirement	Where it is covered
✓	**Primary keys and foreign keys for all relevant tables**	Section 2 (types) and Section 3 (every foreign key, with ON DELETE / ON UPDATE rules)
✓	**UNIQUE, CHECK and DEFAULT wherever required**	Section 4 (full inventory) and Section 5 (ALTER script)
✓	**Explain the purpose of the constraints**	Sections 2, 3, 4 ("Purpose" column) and Section 8
✓	**Execute SQL to verify integrity and relationships**	Section 6 (verification queries with results) and Section 7 (negative tests)

2 Constraint Types at a Glance

Type	What it guarantees	Example in this schema
PRIMARY KEY	Identifies each row uniquely; cannot be NULL	Orders.Order_ID; composite (Order_ID, Item_ID) in Order_Item
FOREIGN KEY	A value must exist in the parent table	Orders.Customer_ID refers to Customer.Customer_ID
UNIQUE	No two rows may share the value (NULLs allowed)	Customer.Email, Payment.Order_ID
CHECK	The value must satisfy a condition	Rating BETWEEN 0.0 AND 5.0; Quantity > 0
DEFAULT	Supplies a value when an insert omits the column	Payment_Status = Pending; Quantity = 1
NOT NULL	The column must always have a value	Customer.Name, Orders.Order_Date (declared in Assignment 5)

3 Primary and Foreign Keys (from Assignment 5)

Foreign keys create the relationships. The **action** chosen for deletion decides what happens to child rows when a parent is removed, which is a design decision, not a technical detail.

Foreign key (child)	Parent	ON DELETE	ON UPDATE	Why
Address.Customer_ID	Customer	RESTRICT	CASCADE	A customer with saved addresses cannot be deleted by accident
Menu_Item.Restaurant_ID	Restaurant	RESTRICT	CASCADE	A restaurant with a menu cannot silently lose its items
Orders.Customer_ID	Customer	RESTRICT	CASCADE	Order history must survive; block deletion of customers who ordered
Orders.Restaurant_ID	Restaurant	RESTRICT	CASCADE	Same reasoning for restaurants
Orders (Address_ID, Customer_ID)	Address (Address_ID, Customer_ID)	RESTRICT	CASCADE	Composite key: the address must belong to the same customer as the order
Orders.Delivery_ID	Delivery_Partner	SET NULL	CASCADE	If a partner leaves, the order stays and simply becomes unassigned
Order_Item.Order_ID	Orders	CASCADE	CASCADE	Deleting an order removes its own lines (classroom convenience)

Foreign key (child)	Parent	ON DELETE	ON UPDATE	Why
Order_Item.Item_ID	Menu_Item	RESTRICT	CASCADE	A menu item that appears on past orders cannot be deleted
Payment.Order_ID	Orders	CASCADE	CASCADE	A payment has no meaning without its order

Production note

Deleting orders and payments is rarely acceptable in a real service; completed orders are normally kept (or soft-deleted) for accounting and audit. CASCADE on Order_Item and Payment is used here only to keep the coursework schema easy to reset.

4 Complete Constraint Inventory

Table	Constraint	Type	Rule	Purpose
Customer	uq_customer_email	UNIQUE	Email	One account per e-mail address
Customer	uq_customer_phone	UNIQUE	Phone	One account per phone number
Restaurant	uq_restaurant_phone	UNIQUE	Phone	No two restaurants share a phone (NULL allowed)
Restaurant	(default)	DEFAULT	Rating = 0.0	New restaurants start unrated
Restaurant	ck_restaurant_rating	CHECK	Rating BETWEEN 0.0 AND 5.0	Ratings stay on a 5-point scale
Menu_Item	(default)	DEFAULT	Availability = TRUE	New items are orderable by default
Menu_Item	ck_menu_price_nonnegative	CHECK	Price >= 0.00	No negative prices
Menu_Item	ck_menu_availability	CHECK	Availability IN (0, 1)	Boolean can only be 0 or 1
Delivery_Partner	uq_delivery_phone	UNIQUE	Phone	One partner per phone
Delivery_Partner	uq_delivery_vehicle	UNIQUE	Vehicle_Number	One partner per vehicle
Delivery_Partner	(default)	DEFAULT	Status = 'Available'	New partners start Available
Delivery_Partner	ck_delivery_status	CHECK	Status IN (Available, Busy, Offline)	Only known statuses
Orders	(default)	DEFAULT	Order_Date = CURRENT_TIMESTAMP	Orders are time-stamped automatically
Orders	(default)	DEFAULT	Order_Status = 'Placed'	Every order starts as Placed
Orders	ck_order_status	CHECK	Status IN (Placed, Confirmed, Preparing, Out for Delivery, Delivered, Cancelled)	Restricts the order life-cycle to six states
Order_Item	(default)	DEFAULT	Quantity = 1	A line without a quantity means one unit
Order_Item	ck_order_item_quantity	CHECK	Quantity > 0	No zero or negative quantities
Order_Item	ck_order_item_unit_price	CHECK	Unit_Price >= 0.00	No negative price snapshots
Payment	uq_payment_order	UNIQUE	Order_ID	At most one payment per order (1 : 0..1)
Payment	(default)	DEFAULT	Payment_Status = 'Pending'	Payments start Pending
Payment	ck_payment_method	CHECK	Method IN (Cash, UPI, Card, Wallet, Net Banking)	Only supported methods
Payment	ck_payment_status	CHECK	Status IN (Pending, Paid, Failed, Refunded)	Only valid payment states
Payment	ck_payment_date_for_completed	CHECK	Status NOT IN (Paid, Refunded) OR Payment_Date IS NOT NULL	A completed payment must carry its date

PRIMARY KEY and FOREIGN KEY definitions are already in Assignment 5. uq_address_owner (Address_ID, Customer_ID) is an extra UNIQUE key created there only so the composite foreign key from Orders has something to point to.

5 The Constraint Script

Run after the Assignment 5 script and **before** inserting the sample data.

```
-- Assignment 6: Implementation of Data Integrity Constraints
-- Run after Assignment_5_Database_Creation.sql, before inserting sample data.
-- Requires MySQL 8.0.16+ for enforced CHECK constraints.
USE online_food_delivery;
-- UNIQUE constraints prevent duplicate account/contact/payment identifiers.
ALTER TABLE Customer
    ADD CONSTRAINT uq_customer_email UNIQUE (Email),
    ADD CONSTRAINT uq_customer_phone UNIQUE (Phone);
ALTER TABLE Restaurant
    ADD CONSTRAINT uq_restaurant_phone UNIQUE (Phone),
    ALTER COLUMN Rating SET DEFAULT 0.0,
    ADD CONSTRAINT ck_restaurant_rating CHECK (Rating BETWEEN 0.0 AND 5.0);
ALTER TABLE Menu_Item
    ALTER COLUMN Availability SET DEFAULT TRUE,
    ADD CONSTRAINT ck_menu_price_nonnegative CHECK (Price >= 0.00),
    ADD CONSTRAINT ck_menu_availability CHECK (Availability IN (0, 1));
ALTER TABLE Delivery_Partner
    ADD CONSTRAINT uq_delivery_phone UNIQUE (Phone),
    ADD CONSTRAINT uq_delivery_vehicle UNIQUE (Vehicle_Number),
    ALTER COLUMN Status SET DEFAULT 'Available',
    ADD CONSTRAINT ck_delivery_status CHECK (Status IN ('Available', 'Busy', 'Offline'));
ALTER TABLE Orders
    ALTER COLUMN Order_Date SET DEFAULT CURRENT_TIMESTAMP,
    ALTER COLUMN Order_Status SET DEFAULT 'Placed',
    ADD CONSTRAINT ck_order_status CHECK (Order_Status IN
        ('Placed', 'Confirmed', 'Preparing', 'Out for Delivery', 'Delivered', 'Cancelled'));
ALTER TABLE Order_Item
    ALTER COLUMN Quantity SET DEFAULT 1,
    ADD CONSTRAINT ck_order_item_quantity CHECK (Quantity > 0),
    ADD CONSTRAINT ck_order_item_unit_price CHECK (Unit_Price >= 0.00);
ALTER TABLE Payment
    ADD CONSTRAINT uq_payment_order UNIQUE (Order_ID),
    ALTER COLUMN Payment_Status SET DEFAULT 'Pending',
    ADD CONSTRAINT ck_payment_method CHECK (Payment_Method IN
        ('Cash', 'UPI', 'Card', 'Wallet', 'Net Banking')),
    ADD CONSTRAINT ck_payment_status CHECK (Payment_Status IN ('Pending', 'Paid', 'Failed', 'Refunded')),
    ADD CONSTRAINT ck_payment_date_for_completed CHECK
        (Payment_Status NOT IN ('Paid', 'Refunded') OR Payment_Date IS NOT NULL);
```

6 Verification After Loading Data

These queries run after Assignment 7. Each returns a count that must be **0**. Results below were computed on the sample dataset.

Verification	Found	Expected	Meaning	Result
Duplicate e-mail groups	0	0	No e-mail occurs twice	Pass
Duplicate customer phone groups	0	0	No phone occurs twice	Pass
Ratings outside 0 to 5	0	0	All ratings valid	Pass
Negative menu prices	0	0	All prices valid	Pass
Quantities of zero or less	0	0	All quantities valid	Pass
Orders using another customer's address	0	0	Composite FK works	Pass
Orders pointing to a missing customer	0	0	No orphan orders	Pass
Order lines pointing to a missing order	0	0	No orphan lines	Pass
Orders with more than one payment	0	0	UNIQUE(Order_ID) holds	Pass
Paid or Refunded without a date	0	0	Date rule holds	Pass

Verification	Found	Expected	Meaning	Result
Invalid order statuses	0	0	Status list holds	Pass

```
SELECT COUNT(*) AS Orders_Using_Another_Customers_Address
FROM Orders o JOIN Address a ON o.Address_ID=a.Address_ID
WHERE o.Customer_ID<>a.Customer_ID
```

Result

All eleven checks return 0, so the sample data satisfies every UNIQUE, CHECK and foreign-key rule.

7 Negative Tests: Proving the Rules Reject Bad Data

A constraint is only proven when an invalid statement is **refused**. Run each statement below on its own, *after* loading the sample data. Wrapping each in a transaction and rolling back keeps the database clean even if a statement unexpectedly succeeds.

#	Scenario	Statement	Expected MySQL error	Rule that fires
1	Duplicate e-mail and phone	INSERT INTO Customer (Name, Email, Phone, Password_Hash) VALUES ('Duplicate Test','asha@example.com','9000000001','demo_hash');	1062 Duplicate entry	uq_customer_email
2	Rating above 5	INSERT INTO Restaurant (Restaurant_Name, Phone, Address, Rating) VALUES ('Invalid Rating Test','9000000999','Test Street',6.0);	3819 Check constraint violated	ck_restaurant_rating
3	Zero quantity	INSERT INTO Order_Item (Order_ID, Item_ID, Quantity, Unit_Price) VALUES (1,2,0,180.00);	3819 Check constraint violated	ck_order_item_quantity
4	Paid without a date	INSERT INTO Payment (Order_ID, Payment_Method, Payment_Status, Payment_Date) VALUES (13,'UPI','Paid',NULL);	3819 Check constraint violated (after the setup in the note below)	ck_payment_date_for_completed
5	Second payment for one order	INSERT INTO Payment (Order_ID, Payment_Method, Payment_Status, Payment_Date) VALUES (1,'Cash','Paid',NOW());	1062 Duplicate entry	uq_payment_order
6	Order using another customer's address	INSERT INTO Orders (Customer_ID, Restaurant_ID, Address_ID) VALUES (1, 1, 3);	1452 Cannot add or update a child row	fk_orders_address_owner
7	Unknown order status	INSERT INTO Orders (Customer_ID, Restaurant_ID, Address_ID, Order_Status) VALUES (1, 1, 1, 'Lost');	3819 Check constraint violated	ck_order_status
8	Delete a customer who has orders	DELETE FROM Customer WHERE Customer_ID = 1;	1451 Cannot delete or update a parent row	fk_address_customer / fk_orders_customer (RESTRICT)

Setup for test 4

Order 1 already has a payment, so a Paid payment for it would hit the UNIQUE rule first. To isolate the CHECK rule, first create a spare order with no payment: INSERT INTO Orders (Customer_ID, Restaurant_ID, Address_ID) VALUES (1, 1, 1); It receives Order_ID 13 (AUTO_INCREMENT continues after 12) and also demonstrates the DEFAULT values. Remove it afterwards with DELETE FROM Orders WHERE Order_ID = 13;

Positive test: DEFAULT values in action

```
START TRANSACTION;
INSERT INTO Orders (Customer_ID, Restaurant_ID, Address_ID) VALUES (1, 1, 1);
SELECT Order_ID, Order_Date, Order_Status      -- Order_Status = 'Placed', Order_Date = now
FROM   Orders ORDER BY Order_ID DESC LIMIT 1;
ROLLBACK;                                     -- leave the sample data unchanged
```

Honest status of these tests

The expected error numbers are the standard MySQL 8 codes for each failure. These eight statements have **not** been executed in this document (no MySQL server was available); run them in Workbench and keep the screenshots as evidence for submission.

8 Why These Constraints Matter, and Their Limits

- **Keys** preserve identity and links, so there are no orphaned lines or payments.
- **UNIQUE** prevents ambiguous accounts and double payments.
- **CHECK** blocks values the business process cannot accept (negative prices, impossible ratings, unknown statuses).
- **DEFAULT** supplies standard starting states and removes repetitive boilerplate.
- **Limit.** A CHECK constraint sees only its own row. Rules that span tables, such as "an order line must come from the order's restaurant" or "never assign an Offline delivery partner", need composite keys (see Assignment 3), triggers or application transactions.
- **Limit.** Constraints do not replace application validation, transaction handling or access control; they are the last line of defence.